

Ticket 001 Incident Report

SSH Authentication Investigation

CyberOps Defense Group Professional Development Program

Investigation type: Authorized cybersecurity lab exercise

Environment: Windows 11, Kali Linux, Ubuntu Linux, and Wazuh

Report generated: July 23, 2026

This report documents a controlled SSH authentication investigation performed in an isolated lab. No production systems, customer information, or third-party accounts were involved.

Executive Summary

Ticket 001 focused on repeated SSH password failures against an Ubuntu server. The activity originated from the authorized Kali Linux lab system at 192.168.56.111 and targeted the sshlab account.

The investigation confirmed 11 failed password attempts followed by a successful login from the same source. After authentication, the user created /tmp/ticket001-access.txt , confirming that activity occurred on the server after access was gained.

I reached this conclusion by comparing native Ubuntu authentication logs, failed- and successful-login history, post-login file metadata, Wazuh alerts, screenshots, and containment records. Using multiple sources made it possible to validate the sequence without depending on a single alert or log file.

After confirming the successful login, I locked the affected account and tested a temporary nftables rule that blocked the source IP. I also developed a Bash detector for repeated SSH authentication failures and scheduled it with cron.

Overall conclusion: Repeated authentication failures were followed by successful access and confirmed post-login activity. The event was contained, documented, and used to test an additional automated detection control.

Incident Summary

Category	Finding
Target account	sshlab
Source system	Kali Linux - 192.168.56.111
Target system	Ubuntu Linux - 192.168.56.121
Failed attempts	11 SSH password failures
Successful access	Confirmed from the same source IP
Post-login activity	Creation of /tmp/ticket001-access.txt
Containment	Account lock and temporary source-IP block
Detection	Wazuh validation and custom Bash detector
Evidence integrity	19 SHA-256 validation checks passed

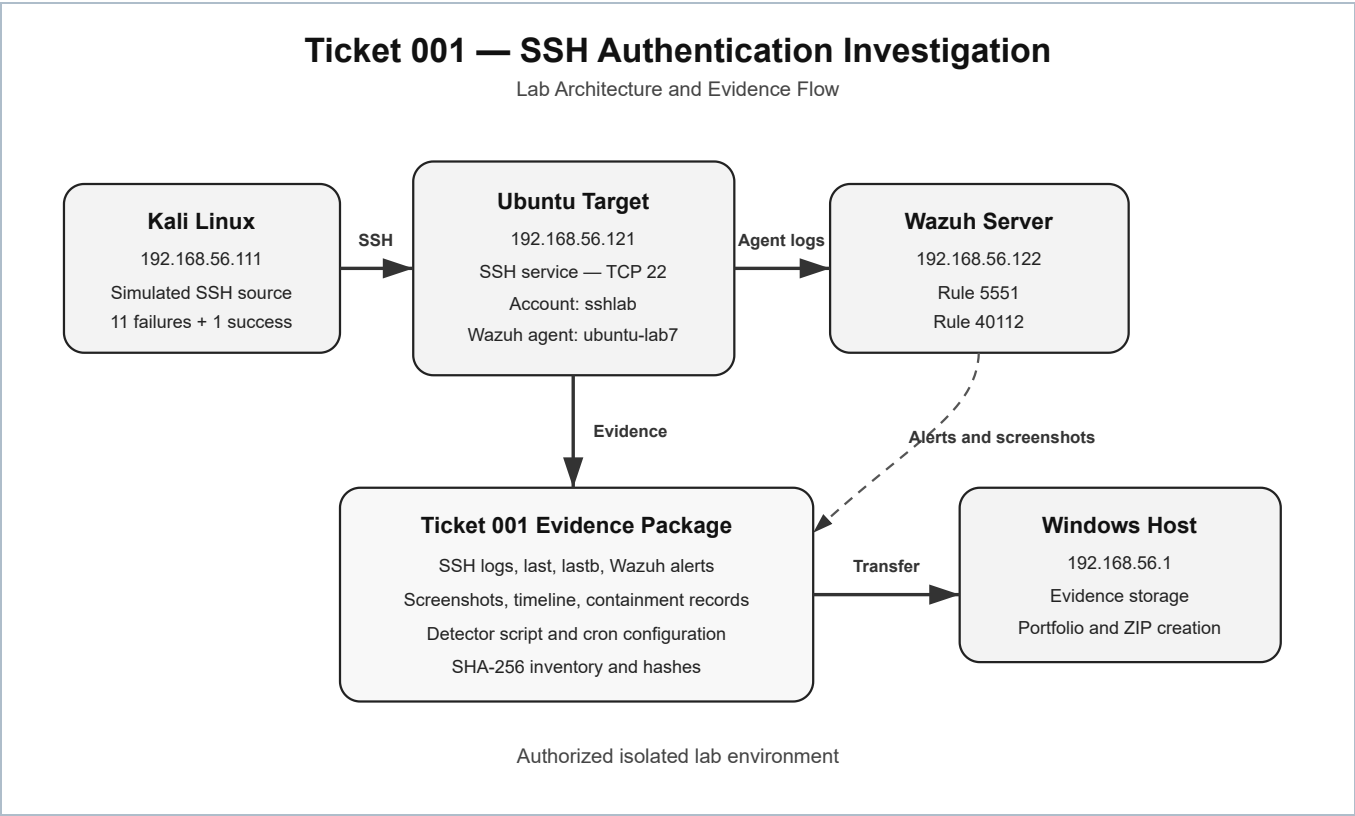
Scope and Environment

The purpose of this investigation was to determine what happened during the SSH authentication sequence, confirm whether access succeeded, identify any post-login activity, review the monitoring response, and

document appropriate containment and detection improvements.

The work was limited to Ticket 001 and was performed entirely in an isolated and authorized lab environment.

System	Role	Lab IP address
Windows 11	Administration, documentation, and evidence storage	192.168.56.1
Kali Linux	Authorized source of the simulated SSH activity	192.168.56.111
Ubuntu Linux	SSH target and Wazuh-monitored endpoint	192.168.56.121
Wazuh server	Central monitoring and alert review	192.168.56.122



Investigation Method

I began with the native Ubuntu records because they were generated directly by the system receiving the SSH connections. This provided a reliable starting point even when the Wazuh agent experienced a temporary connectivity issue.

1. Recorded the investigation start time to separate Ticket 001 activity from older events.
2. Reviewed SSH service records for failed and successful authentication activity.
3. Used `lastb` and `last` to confirm failed-login and successful-session history.
4. Reviewed file metadata to confirm activity after the successful login.
5. Checked Wazuh agent status and repeated controlled testing after connectivity was restored.
6. Compared Wazuh alerts with the findings from the native Ubuntu logs.
7. Applied account-level and temporary network containment.
8. Created and scheduled a custom SSH failure detector.
9. Built an evidence inventory and validated the final SHA-256 manifest.

Why I Chose This Approach

No single record showed the entire incident. The SSH logs showed the authentication sequence, login-history records confirmed the sessions, file metadata demonstrated post-login activity, and Wazuh provided an additional monitoring perspective.

I also preserved the important authentication and session evidence before applying the temporary network block. This reduced the possibility that containment would interrupt the initial fact-finding process.

Detailed Findings

Finding 1 - Repeated authentication failures

The Ubuntu server recorded 11 failed SSH password attempts against the `sshlab` account. The attempts originated from `192.168.56.111`.

Finding 2 - Successful login

A successful SSH login followed the failed attempts. The session came from the same source IP and involved the same targeted account.

Finding 3 - Post-login activity

After authentication, the user created `/tmp/ticket001-access.txt`. The file metadata confirmed that the successful session was used.

Finding 4 - Wazuh detection

Wazuh generated alerts related to the activity after the endpoint agent was confirmed to be connected. Rule 5551 identified repeated authentication failures, while rule 40112 identified multiple failures followed by successful authentication.

Wazuh Detection Evidence

The Wazuh findings supported the native Ubuntu evidence. The temporary agent-connectivity problem also demonstrated why endpoint health and log ingestion should be verified before beginning a detection test.

t	agent.name	ubuntu-lab7
t	data.dstuser	sshlab
t	data.euid	0
t	data.srcip	192.168.56.111
t	data.tty	ssh
t	data.uid	0
t	decoder.name	pam
t	full_log	Jul 20 18:31:31 ubuntu-soc-lab sshd[2804]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=192.168.56.111 user=sshlab
t	id	1784574444.24219
t	input.type	log
t	location	journald
t	manager.name	wazuh-server
t	predecoder.hostname	ubuntu-soc-lab
t	predecoder.program_name	sshd
t	predecoder.timestamp	Jul 20 18:31:31
t	previous_output	> Jul 20 18:31:23 ubuntu-soc-lab sshd[2800]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=192.168.56.111 user=sshlab Jul 20 18:31:27 ubuntu-soc-lab sshd[2802]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=192.168.56.111 user=sshlab Jul 20 18:31:19 ubuntu-soc-lab sshd[2798]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=192.168.56.111 user=sshlab Jul 20 18:31:15 ubuntu-soc-lab sshd[2796]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=192.168.56.111 user=sshlab Jul 20 18:31:08 ubuntu-soc-lab sshd[2794]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=192.168.56.111 user=sshlab Jul 20 18:31:04 ubuntu-soc-lab sshd[2792]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=192.168.56.111 user=sshlab Jul 20 18:30:58 ubuntu-soc-lab sshd[2790]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=192.168.56.111 user=sshlab
t	rule.description	PAM: Multiple failed logins in a small period of time.
#	rule.firedtimes	1
#	rule.frequency	8
t	rule.gdpr	IV_35.7.d, IV_32.2
t	rule.gpg13	7.8
t	rule.groups	pam, syslog, authentication_failures
t	rule.hipaa	164.312.b
t	rule.id	5551
#	rule.level	10
Q	rule.mail	false
t	rule.mitre.id	T1110

Figure 2. Wazuh alert showing repeated SSH authentication failures.

@timestamp	Jul 20, 2026 @ 15:07:25.035
_index	wazuh-alerts-4.x-2026.07.20
agent.id	001
agent.ip	192.168.56.121
agent.name	ubuntu-lab7
data.dstuser	sshlab
data.srcip	192.168.56.111
data.srcport	35618
decoder.name	sshd
decoder.parent	sshd
full_log	Jul 20 18:36:50 ubuntu-soc-lab sshd[2863]: Accepted password for sshlab from 192.168.56.111 port 35618 ssh2
id	1784574445.36507
input.type	log
location	journald
manager.name	wazuh-server
predecoder.hostname	ubuntu-soc-lab
predecoder.program_name	sshd
predecoder.timestamp	Jul 20 18:36:50
rule.description	Multiple authentication failures followed by a success.
rule.firedtimes	1
rule.frequency	2
rule.gdpr	IV_35.7.d, IV_32.2
rule.gpg13	7.1, 7.8
rule.groups	syslog, attacks
rule.hipaa	164.312.b
rule.id	40112
rule.level	12
rule.mail	true

Figure 3. Wazuh alert showing failed attempts followed by successful authentication.

Containment and Detection Improvements

Account Containment

After confirming the successful login, I locked the `sshlab` account. This reduced the possibility that the same credentials could be reused while the investigation continued.

Network Containment

I tested a temporary `nftables` rule against the source IP. The rule blocked additional SSH access as expected. It was later removed so authorized lab testing could continue.

The firewall test reinforced the need to document the expected effect, validation method, and rollback command before applying a temporary control.

Custom Detection

I created a Bash script that reviewed recent SSH authentication failures, counted failures by source IP, compared the count with a defined threshold, and generated an alert when the threshold was reached.

The detector was scheduled with `cron` and produced the expected alert output. Testing also exposed an important limitation: the script did not remember which events it had already processed. Because it searched a rolling time window, the same activity could generate more than one alert.

The detector should therefore be treated as a working proof of concept rather than a production-ready control.

Production improvements needed

- Persistent event state
- Duplicate-alert suppression
- Configurable thresholds and time windows
- Structured alert output
- Improved error handling
- Centralized alert delivery
- Automated testing

Evidence Supporting the Conclusion

Evidence	Purpose
complete-ssh-incident-log.txt	Preserves the broader SSH authentication sequence
failed-logins.txt	Documents repeated password failures
failed-attempts-by-ip.txt	Summarizes failures by source address
lastb-failures.txt	Confirms failed-login history
successful-login-session.txt	Documents the successful SSH session
last-successful-login.txt	Confirms successful-login history
post-login-artifact.txt	Preserves metadata for the file created after login
wazuh-alert-findings.txt	Records relevant Wazuh alert results
Wazuh screenshots	Provide visual confirmation of dashboard detections
Containment records	Document the account lock and temporary firewall rule
Custom detector output	Confirms that the Bash detector generated alerts
technical_timeline.csv	Organizes the important events in sequence
evidence-inventory.txt	Lists the evidence included in the portfolio
evidence-hashes.txt	Provides SHA-256 integrity values

The final validation process recalculated every listed SHA-256 value. All 19 checks passed, and no evidence files were reported as missing or modified.

Technical Challenges and Resolutions

Wazuh Agent Connectivity

The endpoint agent was initially pending or disconnected, creating a temporary monitoring gap.

I continued the investigation using native Ubuntu logs, restored monitoring visibility, and repeated controlled activity to confirm that Wazuh generated the expected alerts.

Firewall Rule Interrupted Testing

The temporary nftables rule successfully blocked the source system, but it also prevented additional authorized SSH testing.

After confirming that the control worked, I removed the rule and continued the remaining lab steps.

Repeated Detector Alerts

The custom detector could alert more than once on the same events because it did not maintain persistent state.

I documented this limitation and identified event-state tracking and duplicate suppression as the primary design improvements.

Windows File-Transfer Path

The first `scp` transfer failed because the Windows destination path ended with a trailing backslash.

I corrected the issue by changing into the destination directory and using a period as the transfer destination.

Alternatives Considered

Relying Only on Wazuh

I considered using only Wazuh alerts as the basis for the investigation. I rejected that approach because the temporary agent-connectivity issue could have left the investigation dependent on incomplete monitoring data.

Native Ubuntu records were used as the primary source, with Wazuh serving as an independent detection and correlation layer.

Blocking the Source Immediately

I could have blocked the source IP before completing the initial evidence review.

Instead, I preserved the important authentication and session records first. This reduced the risk of disrupting the investigation before the main facts were documented.

Continuing With Manual Review Only

Manual review was effective for the one-time investigation, but it would not be efficient as an ongoing monitoring process.

The custom Bash detector provided a repeatable starting point that can be improved with state tracking, structured output, and centralized alerting.

Recommendations

- Confirm Wazuh agent connectivity and log ingestion before beginning security tests.
- Prefer key-based SSH authentication and reduce unnecessary password-based access.
- Review test and privileged accounts regularly and disable accounts that are no longer needed.
- Preserve access to native endpoint logs in addition to centralized monitoring.
- Tune repeated-failure thresholds to match the environment and expected user behavior.
- Add persistent state and duplicate suppression to the custom detector.
- Produce structured detector output for easier ingestion and correlation.
- Document the expected effect, approval process, validation test, and rollback command for containment actions.
- Automate evidence inventory creation, hashing, and validation with one repeatable script.
- Maintain a live technical timeline during future investigations.

Limitations

This investigation was completed in a controlled lab. The results demonstrate the investigation method and behavior of the tested controls, but they do not represent a production compromise.

The custom detector was designed as a proof of concept. It requires additional state management, testing, tuning, and operational integration before production use.

The Wazuh connectivity issue created a temporary monitoring gap. Native Ubuntu evidence reduced the impact of that gap, but the experience showed that monitoring health should be validated before future tests.

Conclusion

The investigation confirmed that 11 failed SSH password attempts against the `sshlab` account were followed by a successful login from the same source system.

The creation of `/tmp/ticket001-access.txt` confirmed activity after authentication.

The conclusion was supported by native Linux logs, login-history records, file metadata, Wazuh alerts, screenshots, containment evidence, custom detector output, and a validated SHA-256 manifest.

The affected account was locked, a temporary source-IP block was tested, and an automated detector was created to improve future monitoring.

The main improvement areas are monitoring-health verification, documented containment rollback, stateful detection logic, duplicate-alert suppression, and greater automation of evidence handling.